

Exam Review

Exam (in-person)

- Open book/notes exam
- You will need to use Google Colab for Python programming
- 150 minutes: **Dec 14, 7:00 – 9:30 pm**
- **COB 122**
- Lecture 1-9

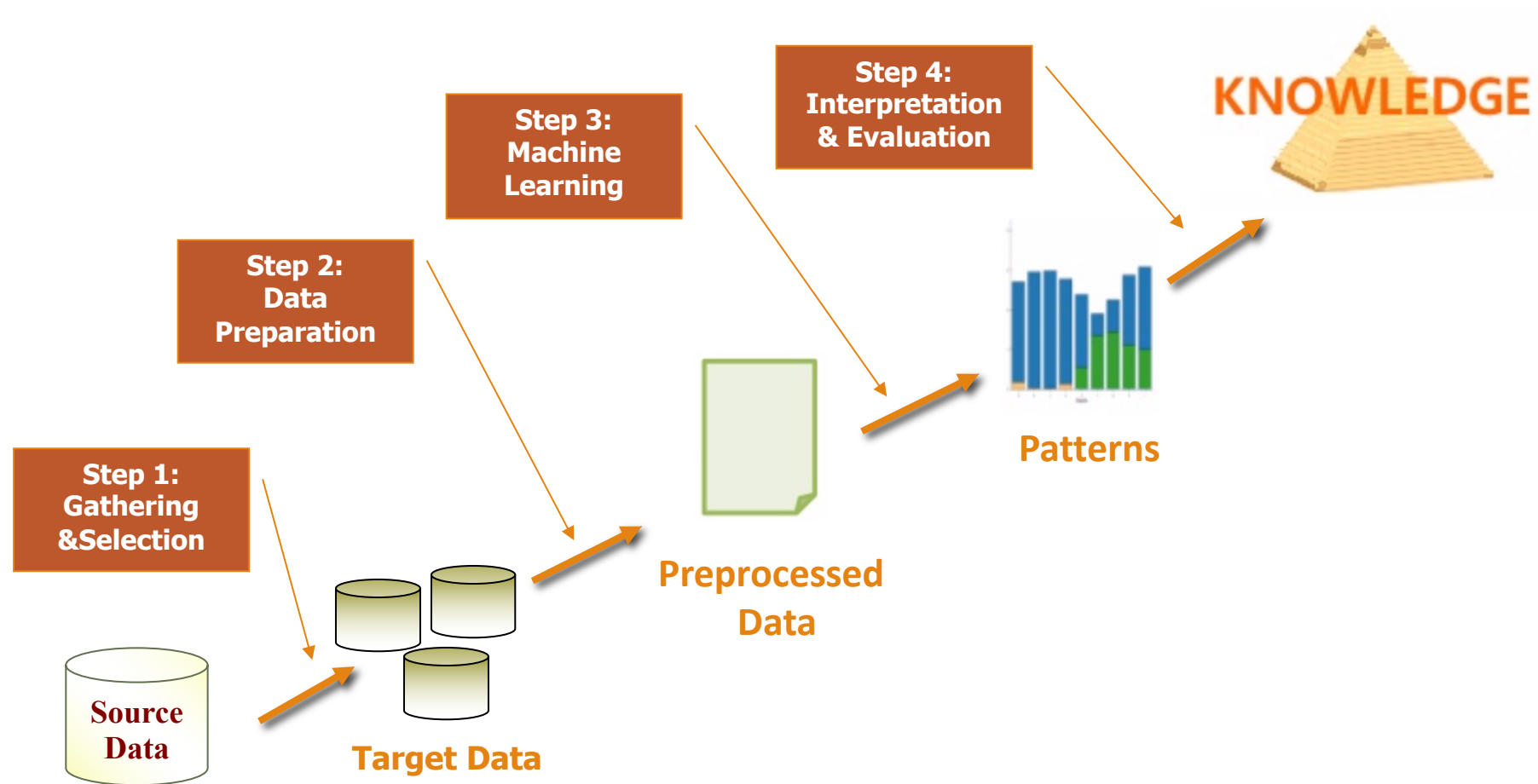
Exam Questions

- 20 questions
 - 15 multiple choices and True false questions (30 points)
 - 4 short answer questions (35 points)
 - Business problem solving
 - Deep learning fundamentals and neural network architecture
 - Convolutional neural network
 - Evaluation results (model fitting, evaluation metrics, model comparison)
- 1 code writing (10 points)

1. Introduction to Deep Learning

- Lecture 1&3 and lab 1&3
- Multiple choices and True False questions

Machine Learning Process



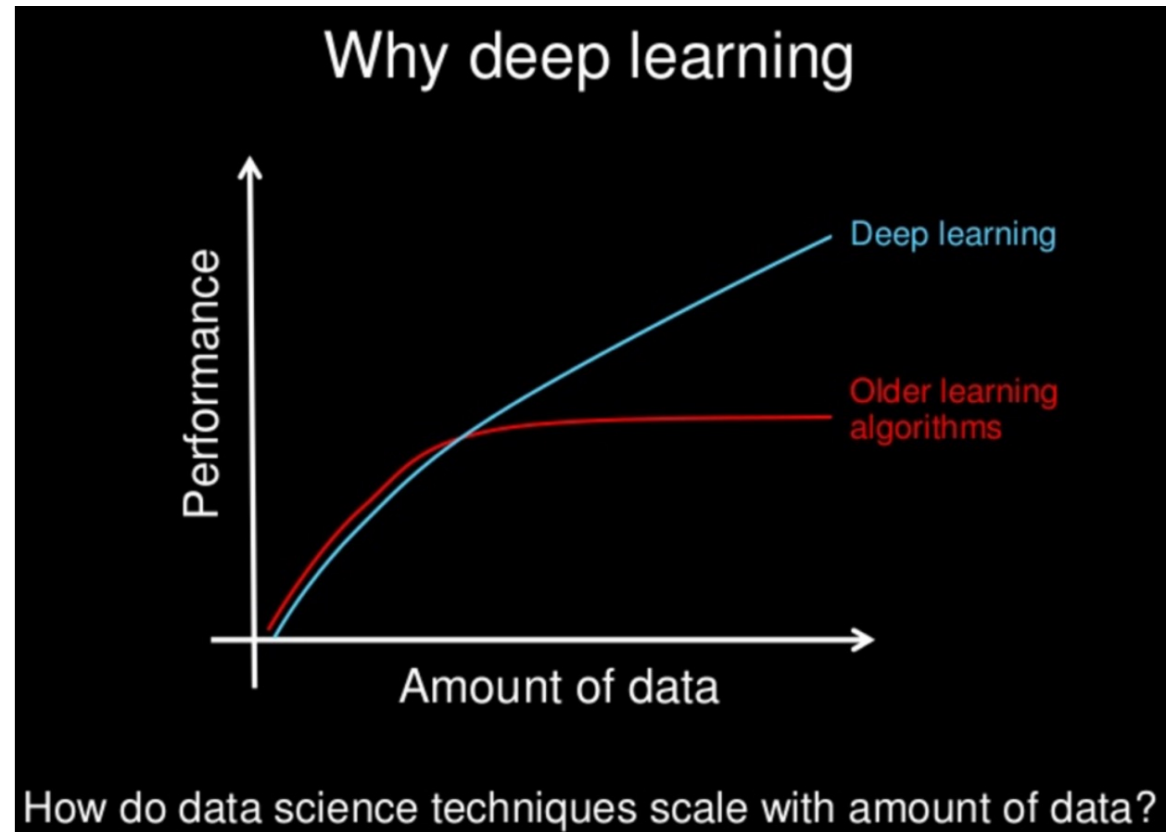
Why Deep Learning

■ Advantages:

- State of the art performance
- Capability of handling unstructured data
- Automatic feature engineering

■ Disadvantages:

- More data
- Bigger models; hard for model training and tuning
- Difficult to interpret
- Resources demanding



Why Deep Learning

- Different types of data

Class Roster			
Course:	MGT 500 Business Policy	Semester:	Spring 2010
Section:	2		
<u>Name</u>	<u>ID</u>	<u>Major</u>	<u>GPA</u>
Baker, Kenneth D.	324917628	MGT	2.9
Doyle, Joan E.	476193248	MKT	3.4
Finkle, Clive R.	548429344	PRM	2.8
Lewis, John C.	551742186	MGT	3.7
McFerran, Debra R.	409723145	IS	2.9
Sisneros, Michael	392416582	ACCT	3.3

Structured data: data stored in tabular form



Unstructured data: image, video, audio, text

Why Deep Learning

- Different types of variables

features

year	model	price	mileage	color	transmission
2011	SEL	21992	7413	Yellow	AUTO
2011	SEL	20995	10926	Gray	AUTO
2011	SEL	19995	7351	Silver	AUTO
2011	SEL	17809	11613	Gray	AUTO
2012	SE	17500	8367	White	MANUAL
2010	SEL	17495	25125	Silver	AUTO
2011	SEL	17000	27393	Blue	AUTO
2010	SEL	16995	21026	Silver	AUTO
2011	SES	16995	32655	Silver	AUTO

examples

If a variable represents a characteristic measured in numbers, it is unsurprisingly called **numeric variable**.

if a variable consists of a set of categories, the variable is called **categorical variable**.

Pretrained Networks

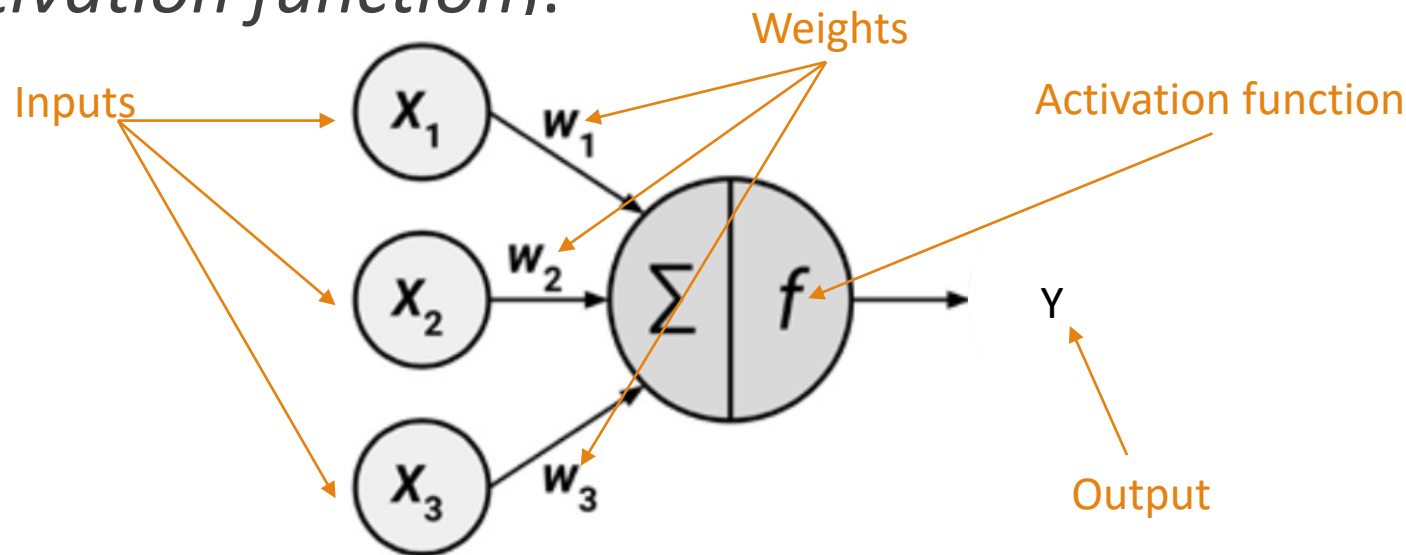
- **Benefits** of pretrained networks:
 - Easy to implement
 - Good performance (as a starting point)
 - Labeled data is not required
 - No model training process
- **Disadvantages:**
 - Not suitable for a different data set or task
 - Different data preprocessing
 - Require transfer and fine tune

2. Deep Learning Fundamentals

- Lecture 4&5 and lab 4&5
- Multiple choices and True False questions
- Short answer question

Artificial Neural Network

- The basic building block of these complicated functions is the **neuron**.
 - A linear transformation of the input (for example, multiplying the input by a number [the *weight*] and adding a constant [the *bias*]) followed by the application of a fixed nonlinear function (referred to as the *activation function*).



$$o = f(w * x + b)$$

Artificial Neural Network

- The simplest unit in (deep) neural networks is a linear operation followed by an activation function.
- The **activation function** plays two important roles
 - It allows the output function to **have different slopes at different values**; neural networks can approximate arbitrary functions
 - At the last layer of the network, it has the role **of concentrating the outputs of the preceding linear operation into a given range** (e.g., Sigmoid and Softmax functions)
- Softmax function:
$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Category	Output	e^{z_i}	Softmax
Reject	-1	0.368	0.047
Approve	2	7.39	0.953

Artificial Neural Network

- **Neural network: a function describing how inputs are mapped to th**

A NEURAL NETWORK

$$O = \tanh(w_n(\dots \tanh(w_2(\tanh(w_1 x + b_1) + b_2)) \dots + b_n))$$

Diagram illustrating a neural network function. The input x is fed into the first layer, which applies the $\tanh$ activation function along with weights w_1 and bias b_1 . This output is then passed through subsequent layers, each applying $\tanh$ activation and weights/biases ($w_2, b_2, \dots, w_n, b_n$). The final output O is the result of the nested $\tanh$ functions. The weights and biases are collectively labeled as LEARNED PARAMETERS.

- **Tensors** refer to the generalization of vectors and matrices to an arbitrary number of dimensions
 - Represent Tabular Data with Tensors: **scalar, vector, matrix, etc.**

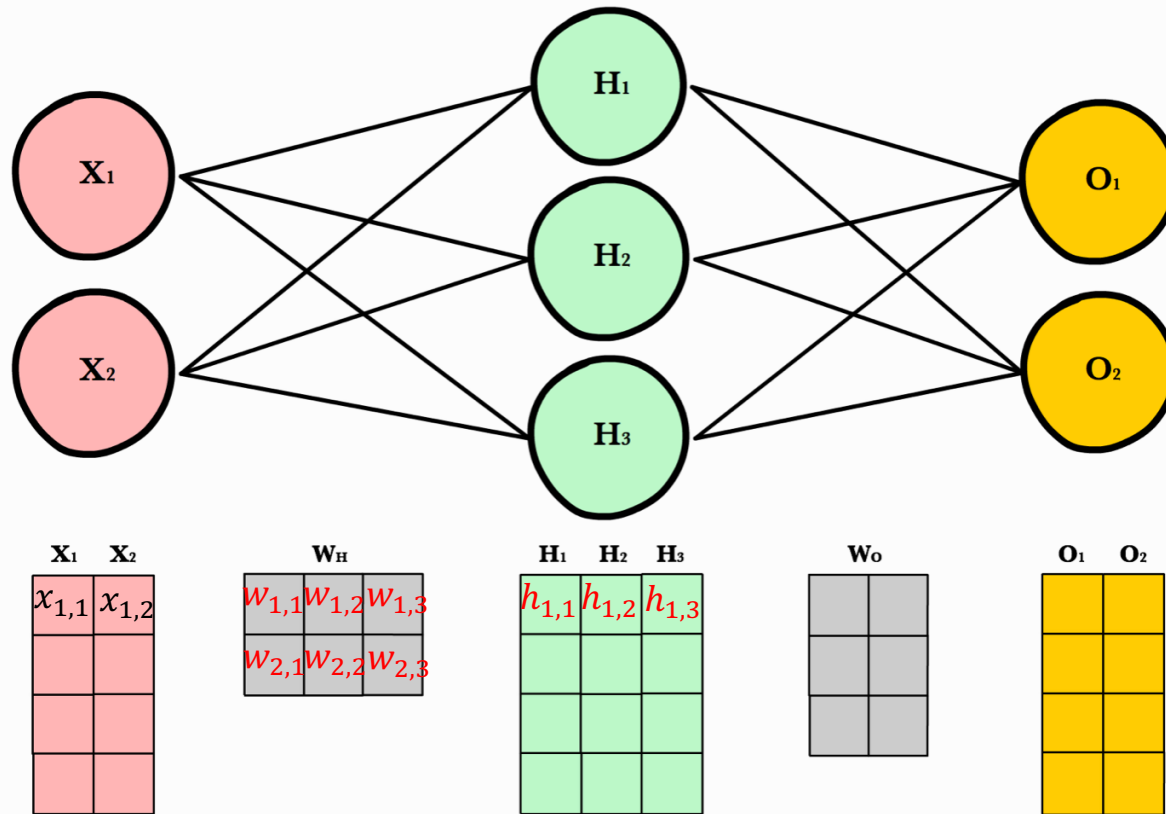
Artificial Neural Network

- A network with 2 input nodes, 3 hidden nodes, and 2 output nodes.

Input layer

Hidden layer

Output layer



$$f(x_{1,1} * w_{1,1} + x_{1,2} * w_{2,1} + b_1) \rightarrow h_{1,1}$$

$$f(x_{1,1} * w_{1,2} + x_{1,2} * w_{2,2} + b_2) \rightarrow h_{1,2}$$

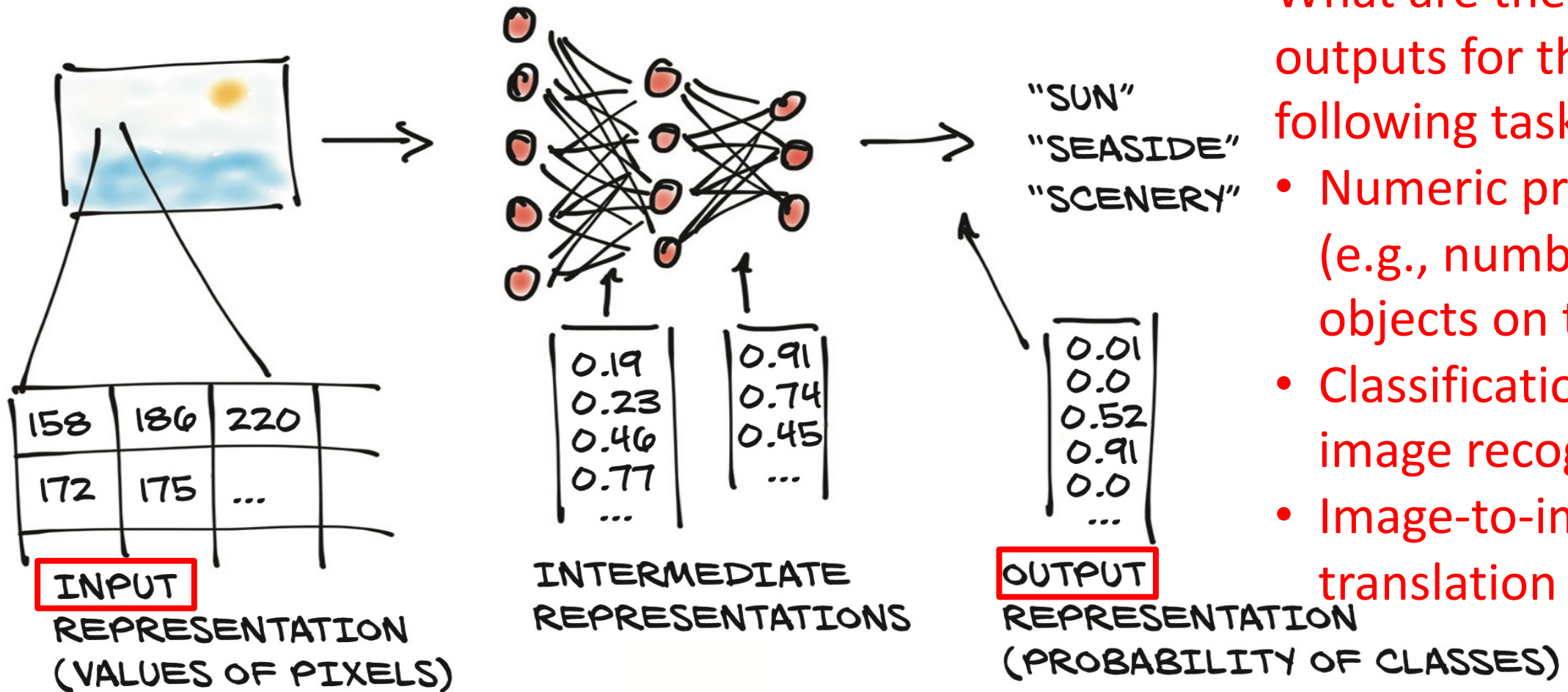
$$f(x_{1,1} * w_{1,3} + x_{1,2} * w_{2,3} + b_3) \rightarrow h_{1,3}$$

$$\mathbf{X} (n \times 2) \quad \mathbf{W} (2 \times 3) \quad \mathbf{H} (n \times 3)$$

3-dimensional hidden space

From <https://ml-cheatsheet.readthedocs.io/en/latest/forwardpropagation.html>

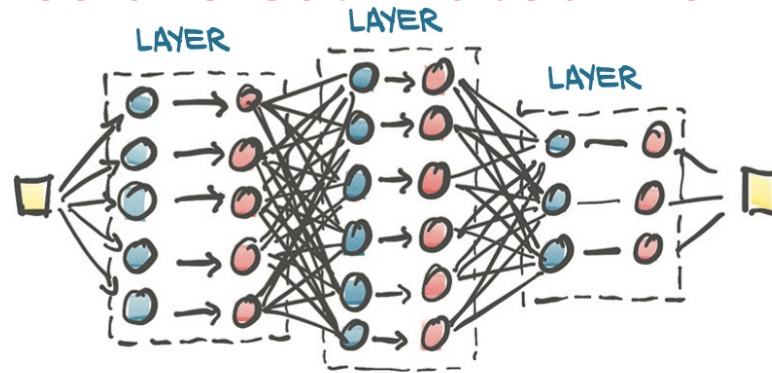
Artificial Neural Network



- What are the model outputs for the following tasks?
- Numeric prediction (e.g., number of objects on the image)
 - Classification (e.g., image recognition)
 - Image-to-image translation

Understand Learning

- **Learn** from data = **fit** the data (by using an algorithm)
 - This process always involves a function with a number of **unknown parameters whose values are estimated from data**: in short, a *model*.



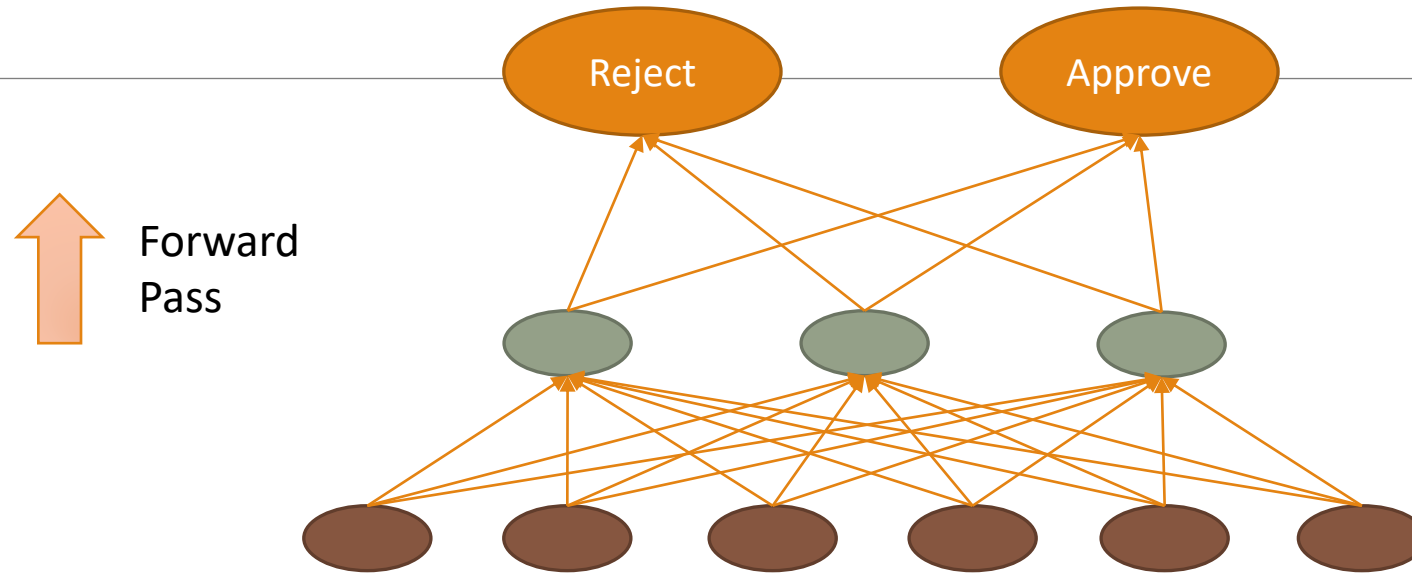
A NEURAL NETWORK

$$\begin{array}{c} \text{OUTPUT} \uparrow \\ 0 = \tanh(w_n(\dots \tanh(w_2(\tanh(w_1 x + b_1) + b_2)) \dots + b_n)) \end{array}$$

↑
LEARNED PARAMETERS

INPUT ↓

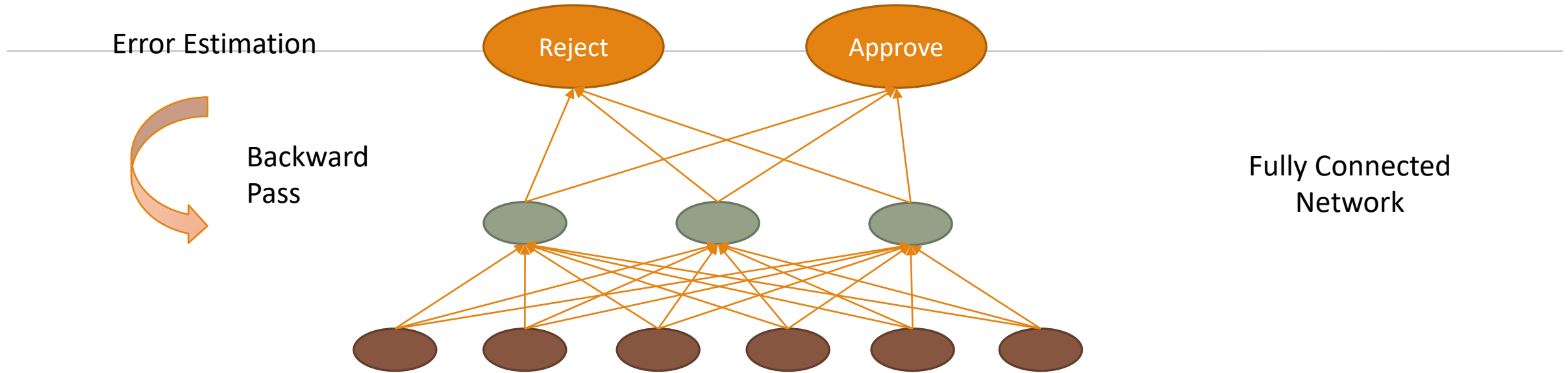
Neural Network: Forward Pass



- Step 1: Model initialization: Randomly set initialized neural network
- Step 2: Feed forward: The model is fed input data

CID	Credit Score	Income (10,000)	Term	Age	Gender	Number of Children	Status
1	6	2	1	35	0	0	Reject
2	7.5	5	3	38	0	2	Approve
3	4	3.5	2	23	1	4	Reject
...							
100000	8	2.5	1	35	0	1	Approve

Neural Network: Backward Pass



- Step 3: Calculate the error: How far is your model output from the actual value.
- Step 4: Compute gradient of the error: apply the chain rule to compute gradient with respect to the parameters
- Step 5: Update the parameters: Update the parameters (weights and biases) until the error becomes minimum.

The Learning Process

- **Forward pass**

- Given input data and initial values for the weights, the model is fed input data

- **Backward pass**

- A measure of the error is evaluated by comparing the outputs to the ground truth
 - The gradient of the error with respect to the parameters is computed using the chain rule
 - Optimize the parameter of the model: the value of the weights is then updated in the direction that leads to a decrease in the error.
- The procedure is repeated until the error falls below an acceptable level.

The Learning Process

Backward pass: 1. A measure of the error is evaluated by comparing the outputs to the ground truth

- **Goal—minimize the error:** Our optimization process should therefore aim at finding w and b so that the loss function is at a minimum (the predicted t_c is close to the actual t_c)
 - We need to exactly define a measure of the error
 - *loss function*—should be high if the error is high and should ideally be as low as possible for a perfect match

The Learning Process

- A **loss function** (or **cost function**) is a function that computes a single numerical value that the learning process will attempt to minimize
 - The calculation of loss typically involves taking the difference between the model outputs and actual values
 - Absolute error loss: $|t_p - t_c|$
 - Squared error loss: $(t_p - t_c)^2$
 - Define the objective of learning

The Learning Process

Backward pass: 2. The gradient of the error with respect to the parameters is computed using the chain rule

- We compute the individual derivatives of the loss with respect to each parameter

```
# Define the gradient function
def grad_fn(t_u, t_c, t_p, w, b):
    dloss_dtp = dloss_fn(t_p, t_c)
    dloss_dw = dloss_dtp * dmodel_dw(t_u, w, b)
    dloss_db = dloss_dtp * dmodel_db(t_u, w, b)
    return torch.stack([dloss_dw.mean(), dloss_db.mean()])
```

→ The gradient depends on both
→ the loss function and model

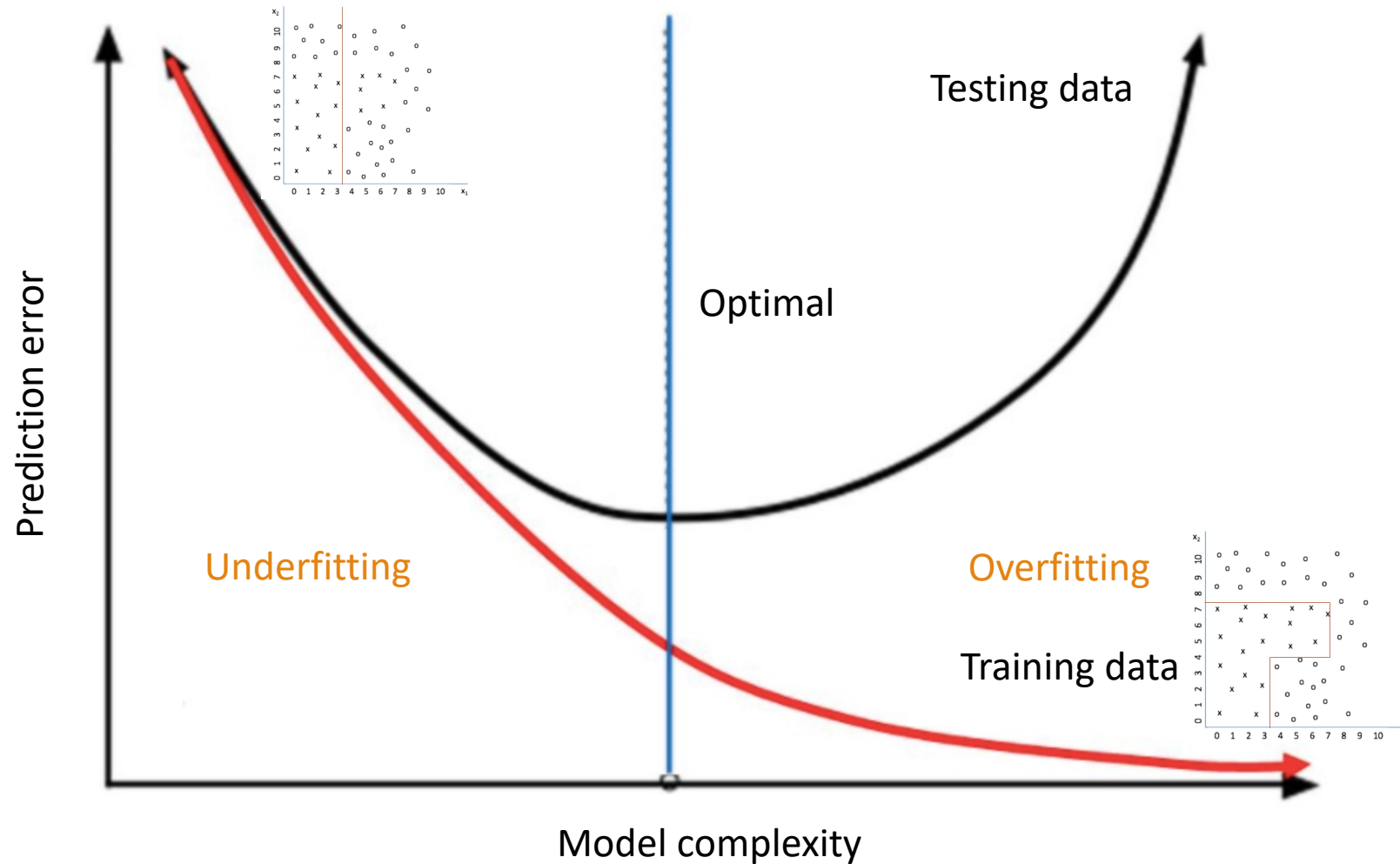
The Learning Process

- **Backward pass:** 3. Optimize the parameter of the model: the value of the weights is then updated in the direction that leads to a decrease in the error.
- **Learning rate:** The learning rate is a hyperparameter that controls how much to change the model in response to the estimated error each time the model weights are updated

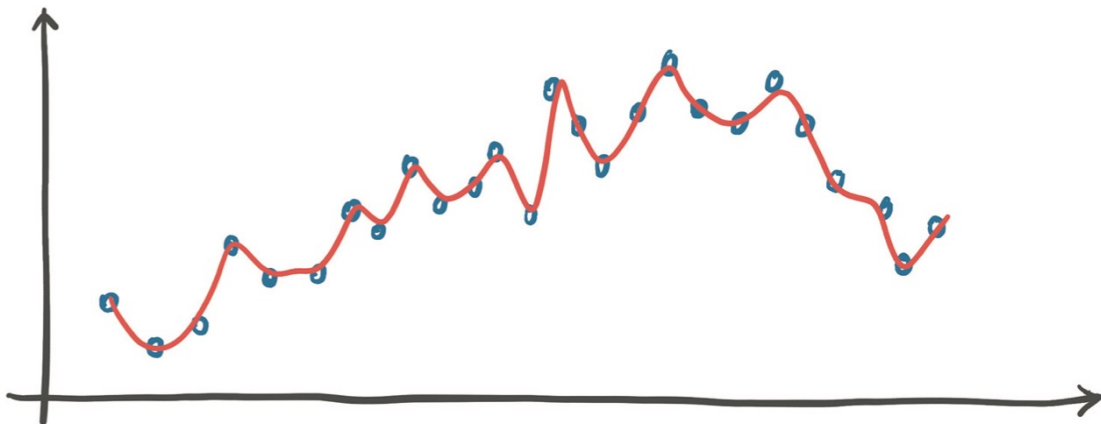
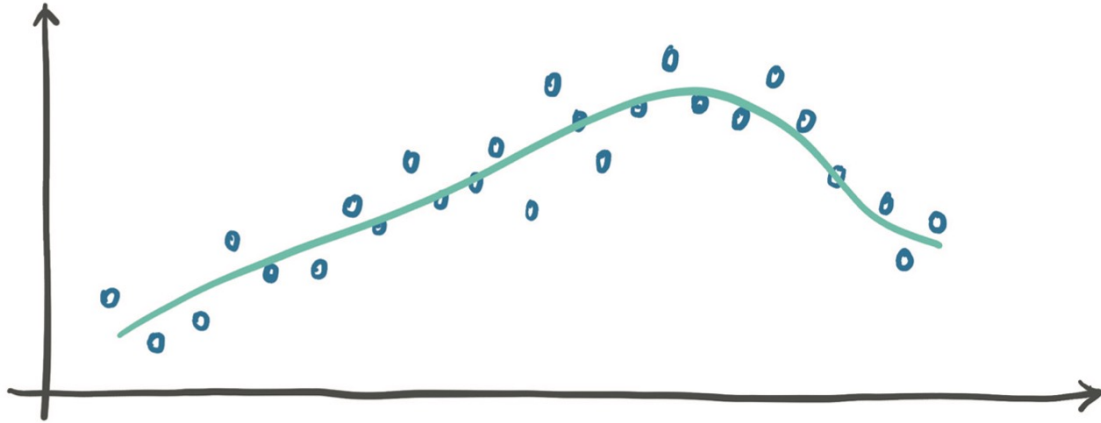
The Learning Process

- **The effect of learning rate**
- **The effect of number of epochs**
- **The effect of hidden layer size and the number of hidden layers**

Generalization and Overfitting



Generalization and Overfitting



- We could have decided to fit the data with a more complicated function, like a really large neural network.
- It could generate a model meandering its way through the data points, just because it pushes the loss very close to zero (Higher chance of overfitting).

Generalization and Overfitting

Causes for Overfitting:

- Training data is not a good representation of testing (new) data
 - Insufficient training data
 - Noises in data: inconsistent class labels for the same values in feature set (input attributes)
 - Outliers in data: the number of samples with a given combination of class labels and feature values is small.
- An algorithm's inability to avoid overfitting noises/outliers or to train generalizable models via small amounts of training data
 - Complex model

Generalization and Overfitting

Avoidance of Overfitting

- Data strategies
 - Secure sufficient data
 - Identify and handle potential outliers and noises
- Evaluation strategies
 - **Identifying overfitting**: comparing the model **performance** on **training** and **testing** data
 - **Avoid big gap between the training and testing accuracy**
- Model strategies
 - Select proper algorithm and manage model complexity
 - Compare different algorithms
 - Lower model complexity via method-specific parameters

3. Structured Data: Classification & Numeric Prediction

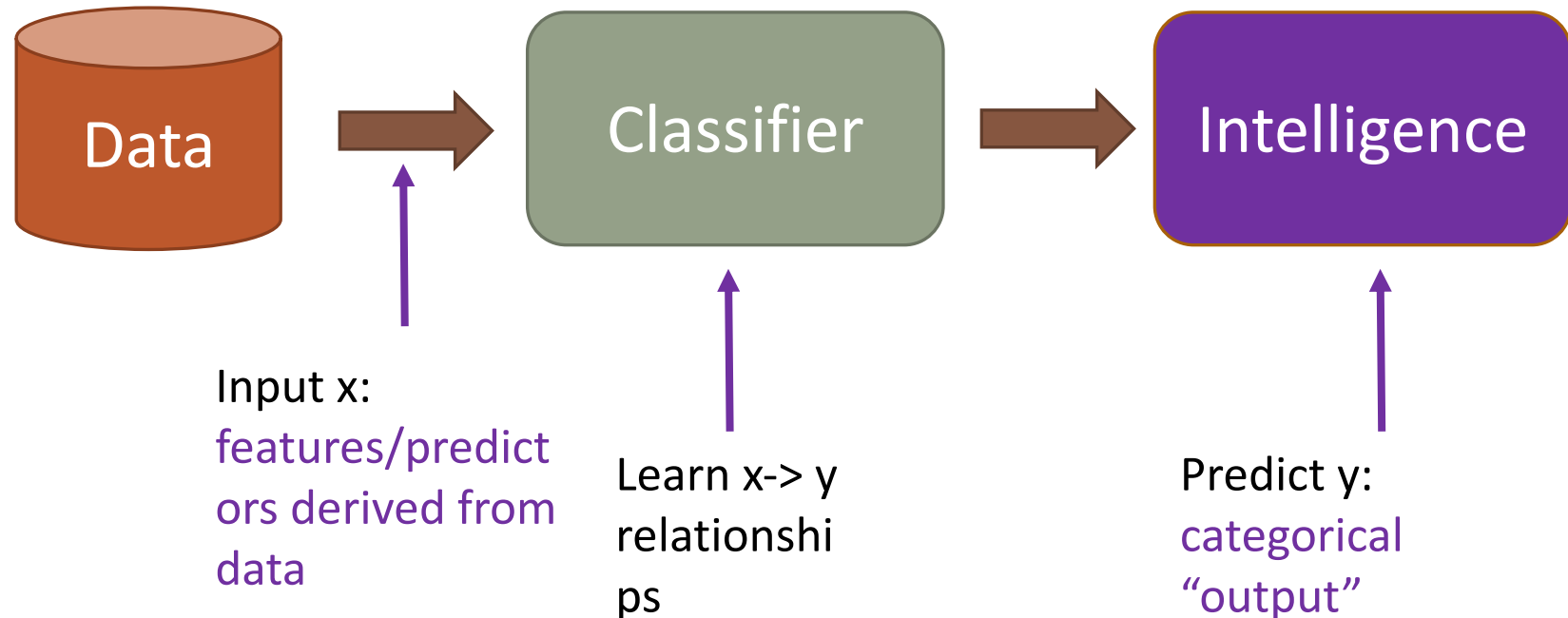
- Lecture 2, 6 and lab 2, 6
- Multiple choices and True False questions
- Short answer question: Evaluation results (model fitting, evaluation metrics, model comparison)

Classification

- Classify objects into a set of **pre-specified** classes (or categories) based on the values of relevant object variables (features).

Why classification?

Estimating/predicting the class or category of action to support **time and cost-effective decision making**.



Classification

- **Predictor:** A predictor (or **predictor variable**) is a variable whose values will be used to predict the value of the target variable.
- **Target variable:** The target variable (or **outcome variable**) is the variable whose values are to be modeled and predicted by other variables.
- **Classifier:** an algorithm/classification model that implements classification (mapping input data to a category).

Steps to build a classification model

- Step 1: Kicked Vehicle Data
 - Data includes both **predictors** and the **target variable**.
- Step 2: Data Preprocessing
 - Categorical variables: **dummy coding**
 - Numeric variables: **Standardization**
 - Split data to **training** and **testing** data
- Step 3: Build a Neural Network on Training Data
- Step 4: Evaluate Neural Network Model Performance

How to preprocess the data
and the purpose of dummy
coding and standardization

The purpose of data splitting

Evaluate Neural Network Model Performance

- Holdout Evaluation

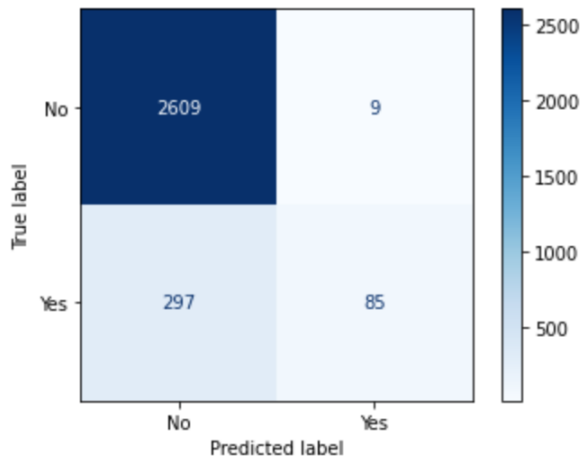
- Splitting method (percentage split)- divide into training and testing sets (e.g. 70%/30% or 2/3 to 1/3)
 - 2 partitions, e.g.
 - 70% and 30% in training and testing
 - Train the model on the **training data**
 - **Training and testing data are not overlapped**

Evaluate Neural Network Model Performance

- **Understand the confusion matrix**
- **Understand the meaning of evaluation metrics**
- **Model comparison**
- **Cost-Benefit analysis**
- **Overfitting issue**

Evaluate Neural Network Model Performance

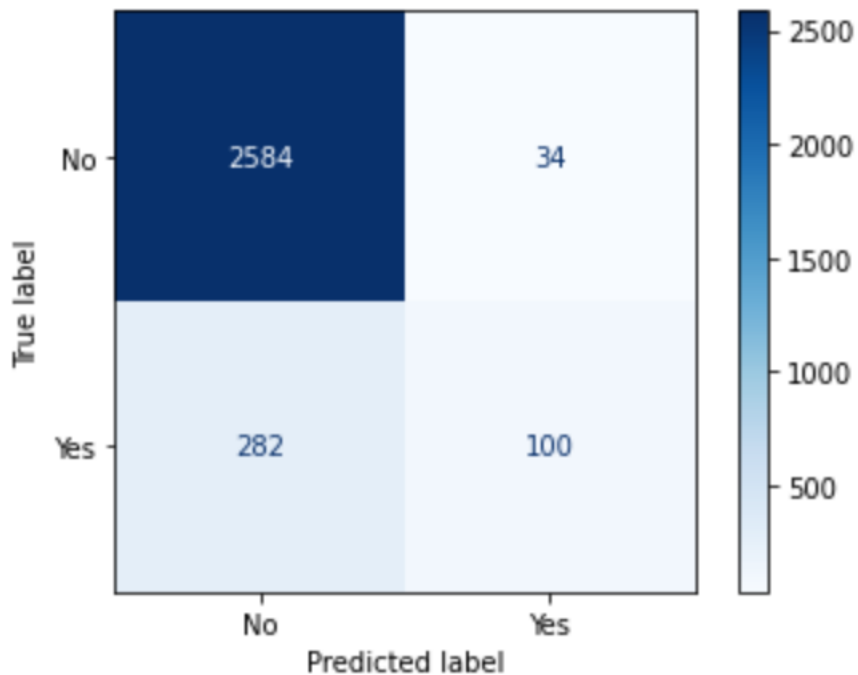
- Precision (**No**) = $TP / (TP + FP) = 2609 / (2609 + 297)$
- Precision (**Yes**) = $TN / (TN + FN) = 85 / (85 + 9)$
- Recall (**No**) = $TP / (TP + FN) = 2609 / (2609 + 9)$
- Recall (**Yes**) = $TN / (TN + FP) = 85 / (85 + 297)$
- F-measure (**a**) = $(2 \times \text{Precision}(\mathbf{a}) \times \text{Recall}(\mathbf{a})) / (\text{Precision}(\mathbf{a}) + \text{Recall}(\mathbf{a}))$
- F-measure (**b**) = $(2 \times \text{Precision}(\mathbf{b}) \times \text{Recall}(\mathbf{b})) / (\text{Precision}(\mathbf{b}) + \text{Recall}(\mathbf{b}))$



	precision	recall	f1-score	support
No	0.90	1.00	0.94	2618
Yes	0.90	0.22	0.36	382
accuracy			0.90	3000
macro avg	0.90	0.61	0.65	3000
weighted avg	0.90	0.90	0.87	3000

Evaluate Neural Network Model Performance

Performance on the **testing data**:



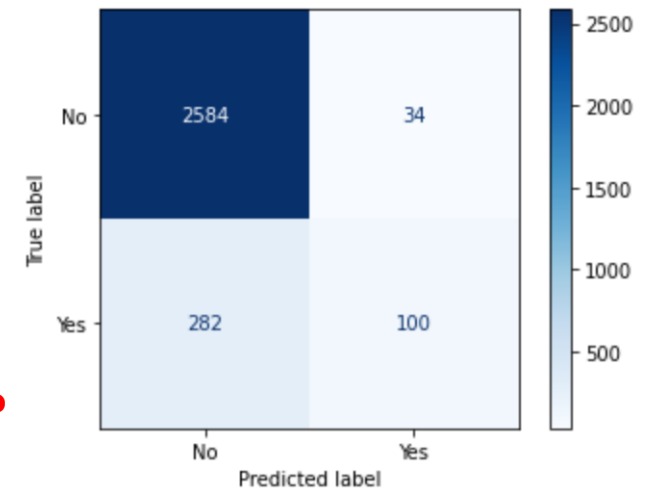
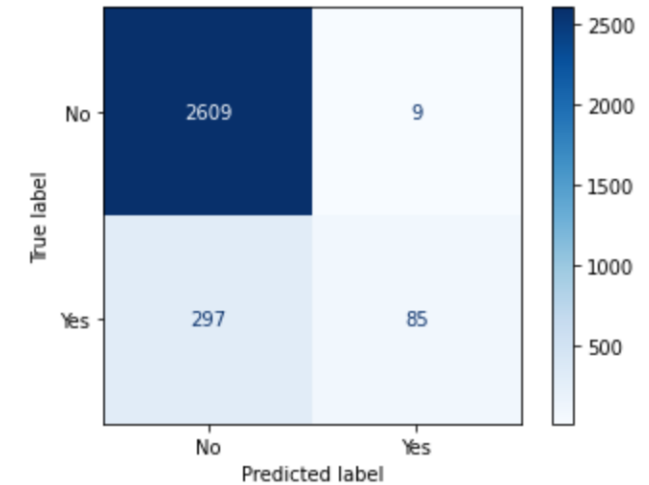
	precision	recall	f1-score	support
No	0.90	0.99	0.94	2618
Yes	0.75	0.26	0.39	382
accuracy			0.89	3000
macro avg	0.82	0.62	0.66	3000
weighted avg	0.88	0.89	0.87	3000

1. How many cars are predicted as Not bad buy?
2. How many bad buy cars are predicted as Not bad buy?
3. How many bad buy cars are identified correctly by the model?
4. If the model predicts a car as bad buy, what is the probability that such prediction is correct?
5. Does the model have better performance on majority (IsBadBuy = 'No') or minority class (IsBadBuy = 'Yes')?

Evaluate Neural Network Model Performance

	precision	recall	f1-score	support
No	0.90	1.00	0.94	2618
Yes	0.90	0.22	0.36	382
accuracy			0.90	3000
macro avg	0.90	0.61	0.65	3000
weighted avg	0.90	0.90	0.87	3000

	precision	recall	f1-score	support
No	0.90	0.99	0.94	2618
Yes	0.75	0.26	0.39	382
accuracy			0.89	3000
macro avg	0.82	0.62	0.66	3000
weighted avg	0.88	0.89	0.87	3000



1. Compare the performance on the majority and minority class
2. Which model is better for identifying BAD BUY cars on testing data
3. Which model has higher confidence in predicting BAD BUY cars on testing data?
4. Which model has better performance on minority class (IsBadBuy = 'Yes')? why?
5. If you are a car dealership, which model you will use, and why?

Evaluate Neural Network Model Performance

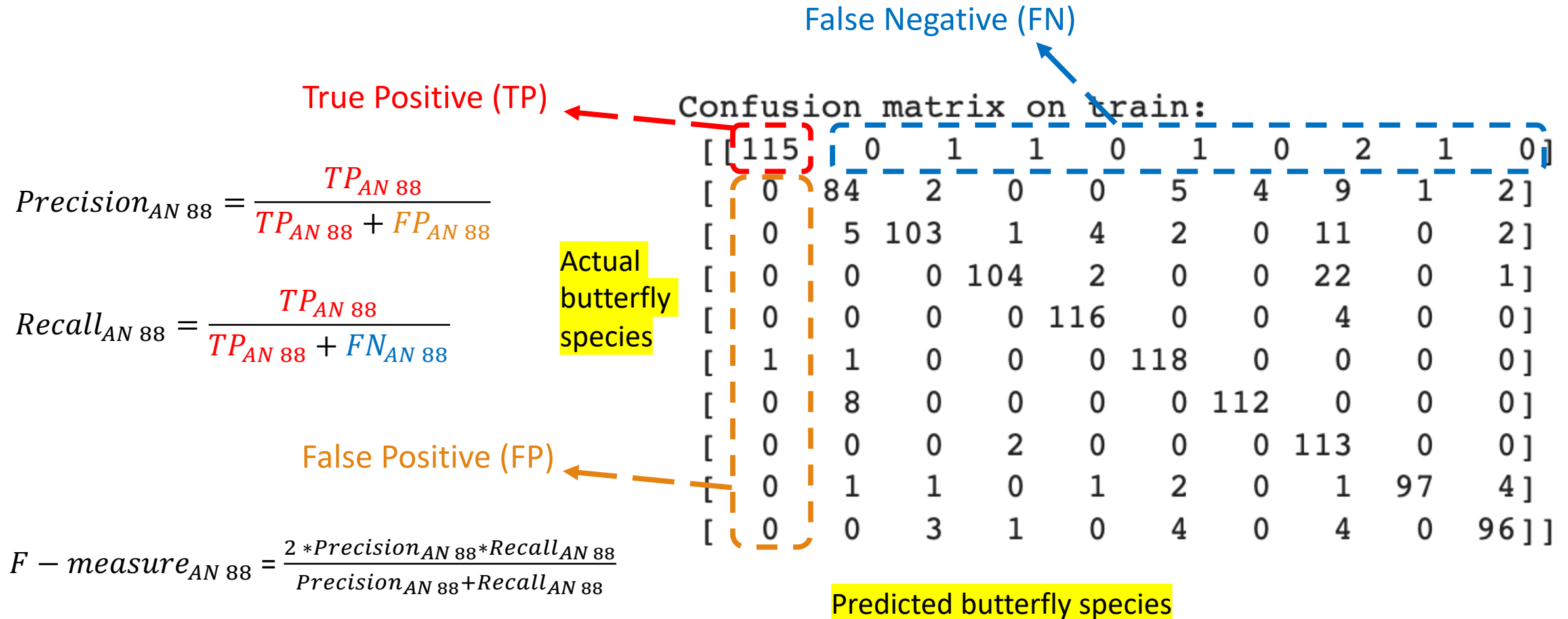
Classification report on train:

precision recall f1-score support

Confusion matrix on train:

[[115	0	1	1	0	1	0	2	1	0]	0	0.99	0.95	0.97	121	
[	0	84	2	0	0	5	4	9	1	2]	1	0.85	0.79	0.82	107
[	0	5	103	1	4	2	0	11	0	2]	2	0.94	0.80	0.87	128
[	0	0	0	104	2	0	0	22	0	1]	3	0.95	0.81	0.87	129
[	0	0	0	0	116	0	0	4	0	0]	4	0.94	0.97	0.95	120
[	1	1	0	0	0	118	0	0	0	0]	5	0.89	0.98	0.94	120
[	0	8	0	0	0	0	112	0	0	0]	6	0.97	0.93	0.95	120
[	0	0	0	2	0	0	0	113	0	0]	7	0.68	0.98	0.80	115
[	0	1	1	0	1	2	0	1	97	4]	8	0.98	0.91	0.94	107
[	0	0	3	1	0	4	0	4	0	96]]]	9	0.91	0.89	0.90	108
											ccuracy			0.90	1175
											macro avg			0.91	1175
											weighted avg			0.91	1175

Evaluate Neural Network Model Performance



Evaluate Neural Network Model Performance

■ Macro average:

$$Precision_{macro} = \frac{1}{10} (Precision_{AN\ 88} + Precision_{blue} + \dots + Precision_{zebra})$$

$$Recall_{macro} = \frac{1}{10} (Recall_{AN\ 88} + Recall_{blue} + \dots + Recall_{zebra})$$

$$F - measure_{macro} = \frac{1}{10} (F - measure_{AN\ 88} + F - measure_{blue} + \dots + F - measure_{zebra})$$

Balanced vs
imbalanced data

■ Weighted average:

$$Precision_{weighted} = \frac{(Precision_{AN\ 88} * N_{AN\ 88} + Precision_{blue} * N_{blue} + \dots + Precision_{zebra} * N_{zebra})}{Total\ number\ of\ samples}$$

$$Recall_{weighted} = \frac{(Recall_{AN\ 88} * N_{AN\ 88} + Recall_{blue} * N_{blue} + \dots + Recall_{zebra} * N_{zebra})}{Total\ number\ of\ samples}$$

$$F - measure_{weighted} = \frac{(F - measure_{AN\ 88} * N_{AN\ 88} + F - measure_{blue} * N_{blue} + \dots + F - measure_{zebra} * N_{zebra})}{Total\ number\ of\ samples}$$

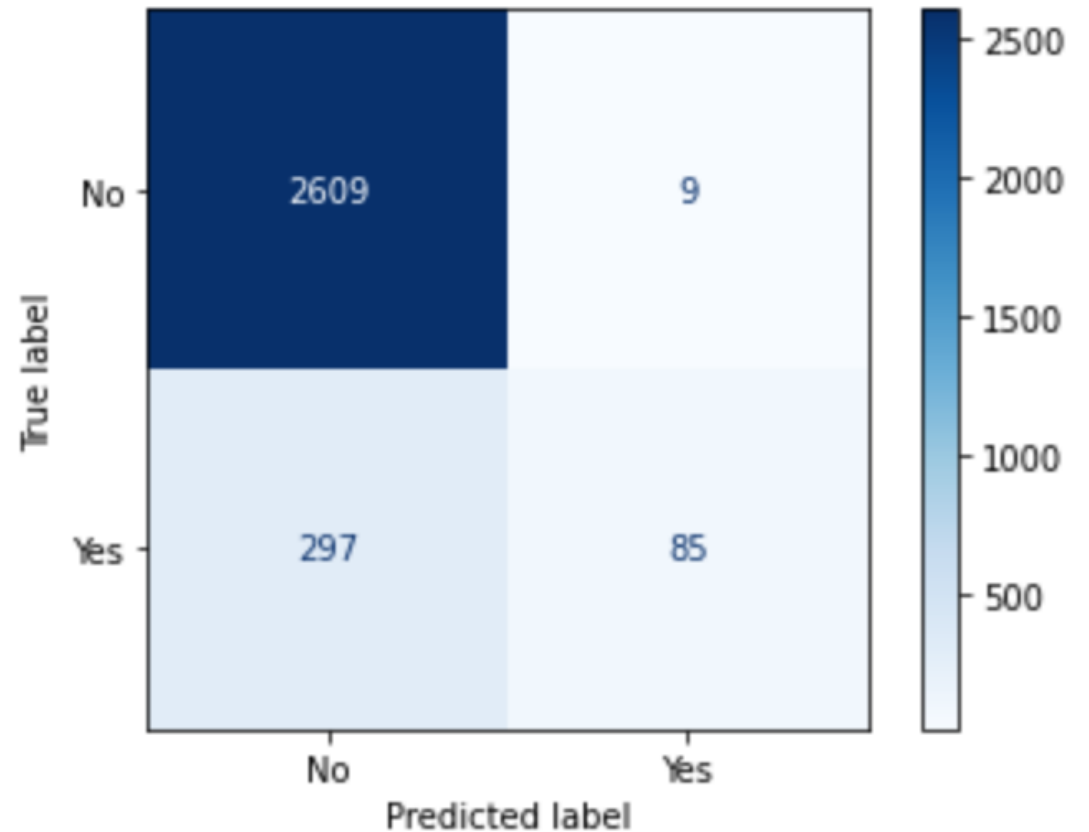
Evaluate Neural Network Model Performance

Assume the following costs / benefits:

- Cost of buying a bad car: \$2000
- Profit of purchasing a good car: \$300
- Opportunity cost of a good car that is missed: \$300

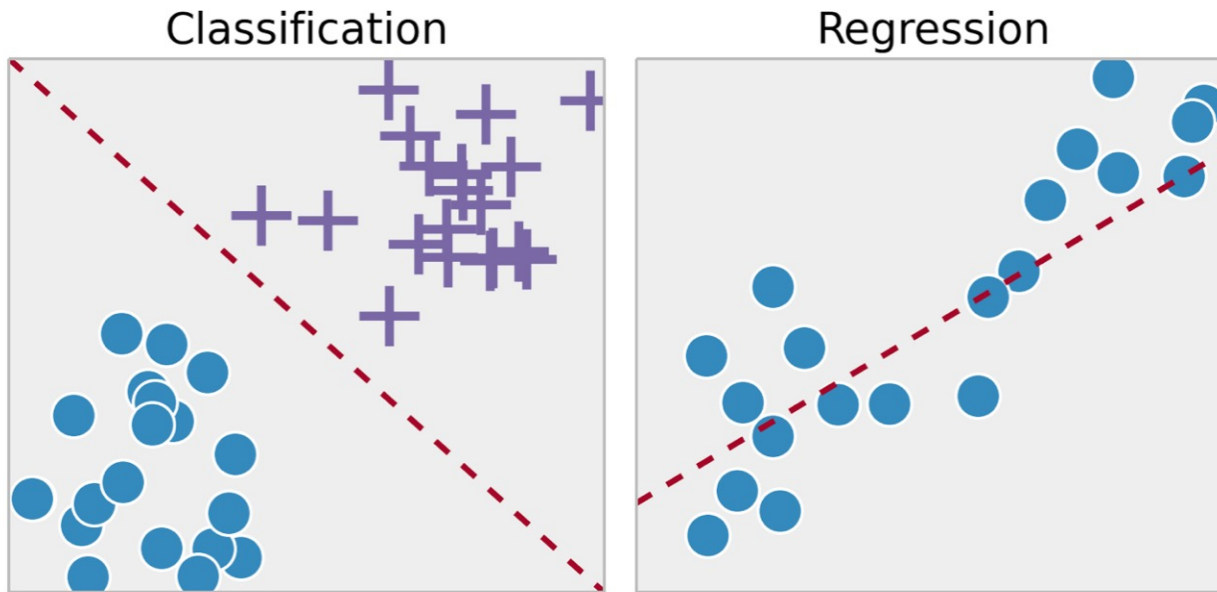
Decision Tree model:

- Cost: $297 * 2000$
- Profit: $2609 * 300$
- Opportunity cost: $9 * 300$
- Profit – Cost - Opportunity cost: 186,000



Numeric Prediction vs Classification

- **Differences between Classification and Numeric Prediction**
 - Target variable
 - Evaluation metrics



Evaluation Metrics for Numeric Prediction

■ Mean Absolute Error (**MAE**)

- The **mean absolute error (MAE)** is a quantity used to measure how close forecasts or predictions are to the eventual outcomes.
- This is known as a scale-dependent accuracy measure and therefore cannot be used to make comparisons between series using different scales

$$\text{MAE} = \frac{1}{n} \sum_{t=1}^n |y_t - \hat{y}_t|$$

■ Root Mean Squared Error (**RMSE**)

- **RMSE** gives a relatively high weight to large errors. This means the RMSE should be more useful when large errors are particularly undesirable.

$$\text{RMSE} = \sqrt{\frac{\sum_{t=1}^n (y_t - \hat{y}_t)^2}{n}}$$

MAE: 4011.3808850910846
RMSE: 5773.7111375700415

4. Recommender Systems

- Lecture 7 and lab 7
- Multiple choices and True False questions

Data for Recommendation

- **Explicit feedback**

- Explicit feedback are provided by the user. They infer the user's preference. Examples include ratings and reviews.

- **Implicit feedback**

- Implicit feedback are provided when users interact with the item. They infer a user's behavior and are easy to get as users are subconsciously clicking. Examples include clicks, views and purchases.

Data for Recommendation

■ Explicit feedback

	<i>Batman Begins</i>	<i>Alice in Wonderland</i>	<i>Dumb and Dumber</i>	<i>Equilibrium</i>
User A	4	?	3	5
User B	?	5	4	?
User C	5	4	2	?
User D	2	4	?	3
User E	3	4	5	?

■ Implicit feedback

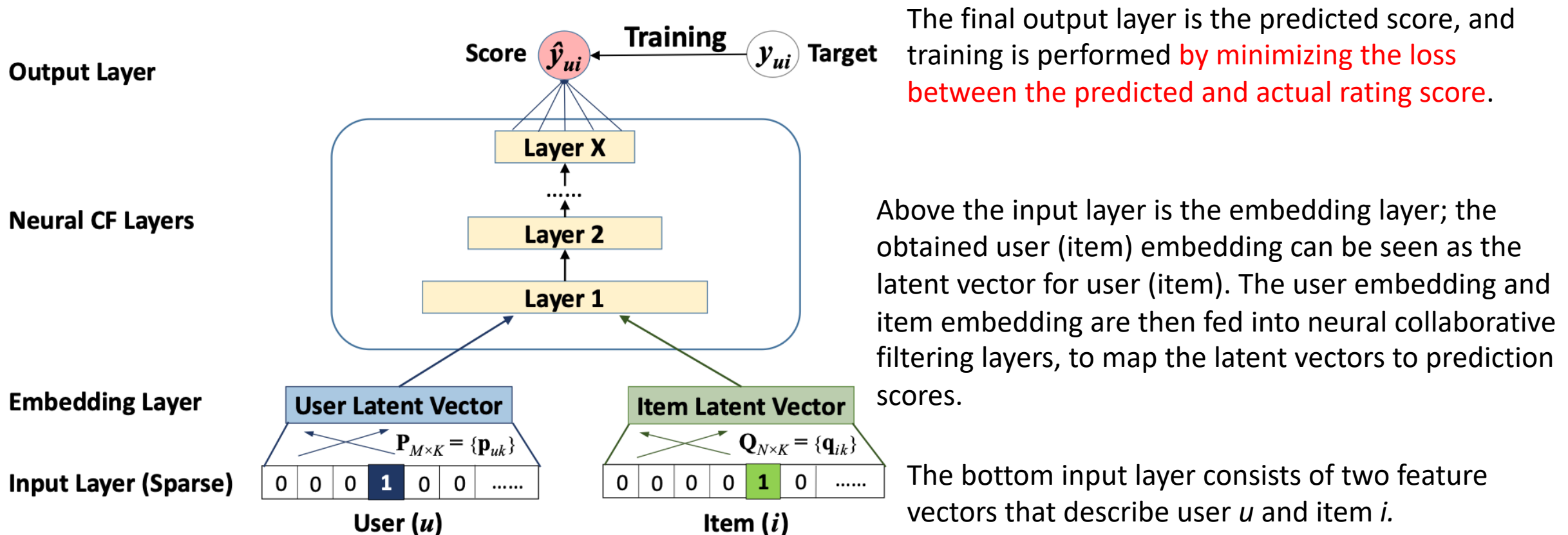
	<i>Item 1</i>	<i>Item 2</i>	<i>Item 3</i>	<i>Item 4</i>
User A	1	?	1	1
User B	?	1	1	?
User C	1	1	1	?
User D	1	1	?	1
User E	1	1	1	?

Steps to Build A Neural Collaborative Filtering

- Step 1: MovieLense Data
- Step 2: Data Preprocessing
- Step 3: Build a Neural Collaborative Filtering on Training Data
- Step 4: Evaluate Neural Collaborative Filtering Performance

Neural Collaborative Filtering

- He, X., et al. Neural collaborative filtering. *Proceedings of the 26th international conference on world wide web*. 2017.



Neural Collaborative Filtering

■ User/item embedding

- Embedding maps users/items to low-dimensional vectors in a way that similar users/items are close to each other.

user	Embedding vector							
1	0.6	0.9	0.1	0.4	-0.7	-0.3	-0.2	0.4
2	0.5	0.8	0.2	0.3	-0.9	-0.1	-0.1	0.4
3	-0.4	-0.5	0.5	0.6	0.8	-0.8	0.2	0.5

- Embedding transform user/item id into numbers (input must be numeric for neural networks)
- Embedding transform learns similarities among users/items

Evaluate Neural Collaborative Filtering Performance

■ Predict ratings

	user_id	movie_id	rating	timestamp
31	0	2969	4	978300019
22	0	1178	5	978300055
27	0	1574	4	978300055
37	0	957	5	978300055
24	0	2147		978300103
...	...	...	...	...
1000019	6039	2709	4	997454429
999988	6039	1741	4	997454464
1000172	6039	1618	3	997454464
1000167	6039	155	3	997454486
1000042	6039	1131		998315055

1000209 rows x 4 columns

Predict ratings

Testing data

Evaluate Neural Collaborative Filtering Performance

■ Mean Absolute Error (MAE)

- The **mean absolute error (MAE)** is a quantity used to measure how close forecasts or predictions are to the eventual outcomes.
- This is known as a scale-dependent accuracy measure and therefore cannot be used to make comparisons between series using different scales

$$\text{MAE} = \frac{1}{n} \sum_{t=1}^n |y_t - \hat{y}_t|$$

■ Root Mean Squared Error (RMSE)

- **RMSE** gives a relatively high weight to large errors. This means the RMSE should be more useful when large errors are particularly undesirable.

$$\text{RMSE} = \sqrt{\frac{\sum_{t=1}^n (y_t - \hat{y}_t)^2}{n}}$$

Training MAE and RMSE: 0.6683394005368883 0.8520005597969494

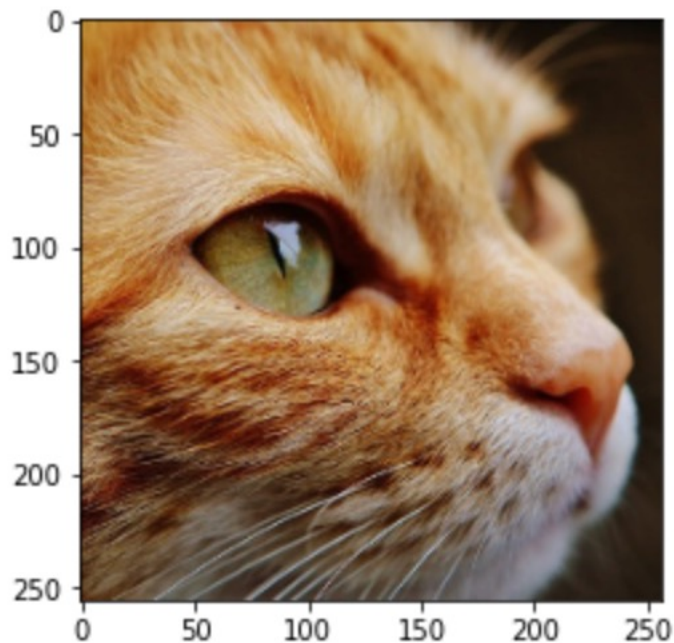
Testing MAE and RMSE: 0.7322674097999042 0.9388386549274153

5. Learning from images

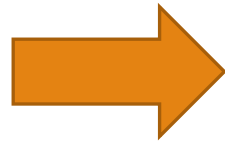
- Lecture 8&9 and lab 8&9
- Multiple choices and True False questions
- Short answer question: convolutional neural network

Represent Image Data with Tensors

- PyTorch modules dealing with image data require tensors to be laid out as $C \times H \times W$: channels, height, and width, respectively.



How to represent the image data



```
tensor([[[[202, 193, ..., 13, 12],  
         [199, 192, ..., 14, 14],  
         ...,  
         [ 75,  68, ..., 36, 37],  
         [ 85, 103, ..., 37, 38]],  
        [[151, 139, ...,  9,  8],  
         [151, 140, ..., 11, 11],  
         ...,  
         [ 33,  26, ..., 26, 27],  
         [ 40,  58, ..., 27, 28]],  
        [[ 68,  53, ...,  6,  5],  
         [ 67,  54, ...,  6,  6],  
         ...,  
         [ 11,   2, ..., 17, 18],  
         [ 19,  37, ..., 18, 19]]], dtype=torch.uint8)
```

$C \times H \times W$: $3 \times 256 \times 256$

Each value is between 0 to 255

Steps to Build A Neural Network for Image Classification

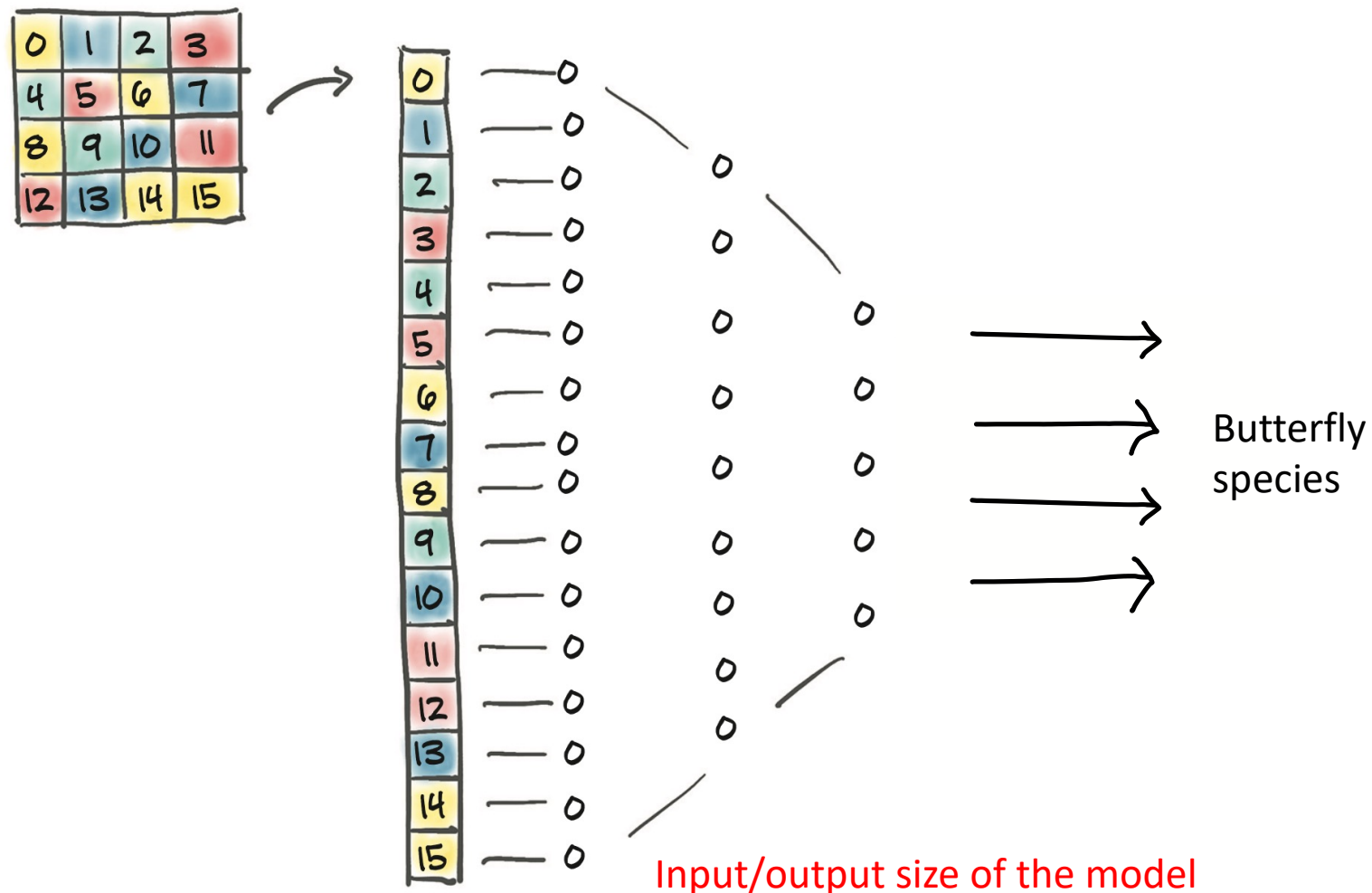
- Step 1: Butterfly Data
- Step 2: Data Preprocessing
 - Resize data
 - Convert to tensors
 - Normalize data (Z-score standardization)
- Step 3: Build a Neural Network on Training Data
 - Fully connected neural network
 - Convolutional neural network
- Step 4: Evaluate Neural Network Performance

How to preprocess the image data and the purpose of resizing data, converting to tensors, and normalization.

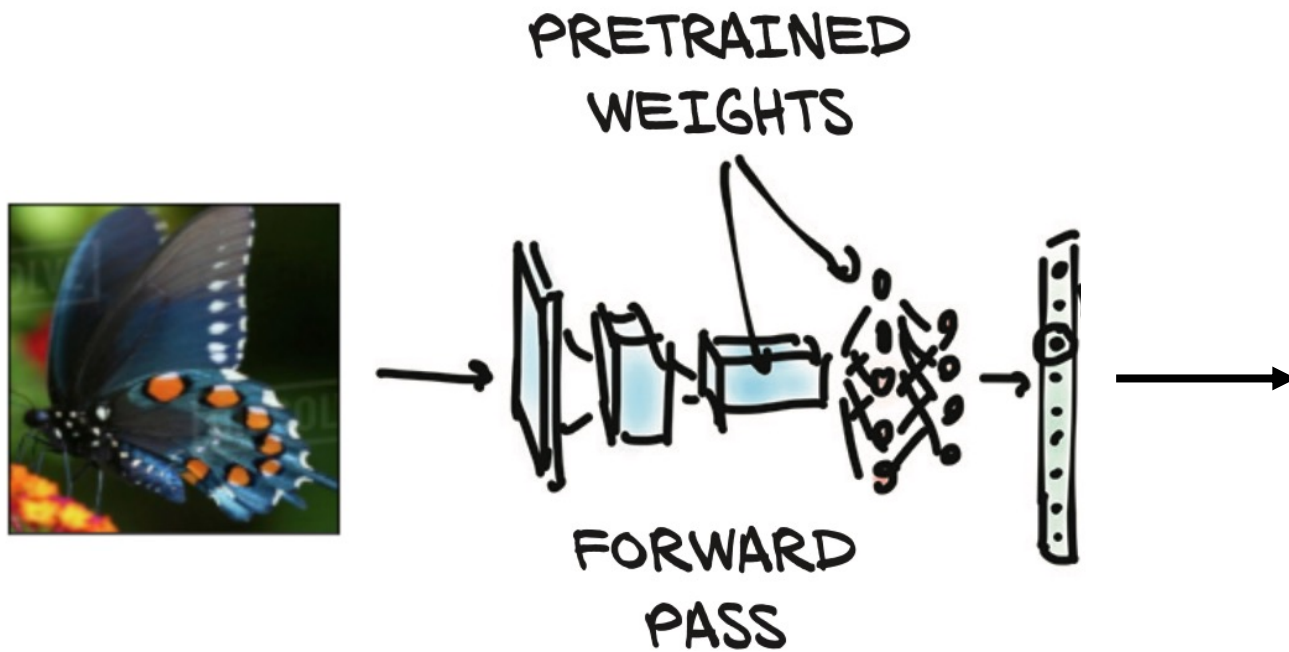
Fully Connected Neural Network on Image Data

- If we just take the image pixels and straighten them into a long 1D vector, we could consider those numbers as input features

- $3 \times 224 \times 224$: that is, 150,528 input features per sample.



Fully Connected Neural Network on Image Data



Score	Butterfly species
y_0	AN 88
y_1	BLUE MORPHO
y_2	COMMON WOOD-NYMPH
y_3	MONARCH
y_4	PEACOCK
y_5	PIPEVINE SWALLOW
y_6	ULYSES
y_7	VICEROY
y_8	YELLOW SWALLOW TAIL
y_9	ZEBRA LONG WING

Each element
between 0 and 1

$$\text{softmax}(y_0, y_1, \dots, y_9) = \left(\frac{e^{y_0}}{e^{y_0} + e^{y_1} + \dots + e^{y_9}}, \frac{e^{y_1}}{e^{y_0} + e^{y_1} + \dots + e^{y_9}}, \dots, \frac{e^{y_9}}{e^{y_0} + e^{y_1} + \dots + e^{y_9}} \right)$$

Sum of element equals 1

Fully Connected Neural Network on Image Data

■ Cross Entropy Loss

- Measures the performance of a classification model by calculating the difference between two probability distributions

- Predicted probability distribution

$$\text{softmax}(y_0, y_1, \dots, y_9) = \left(\frac{e^{y_0}}{e^{y_0} + e^{y_1} + \dots + e^{y_9}}, \frac{e^{y_1}}{e^{y_0} + e^{y_1} + \dots + e^{y_9}}, \dots, \frac{e^{y_9}}{e^{y_0} + e^{y_1} + \dots + e^{y_9}} \right)$$

- Actual probability distribution

PEACOCK $\longrightarrow$ (0, 0, 0, 0, 1, 0, 0, 0, 0, 0)

$$\text{Cross Entropy} = -\frac{1}{N} \sum_j y_j * \log(\hat{y}_j) ; \quad y_j \text{ is the true label and } \hat{y}_j \text{ is the softmax probability}$$

Fully Connected Neural Network on Image Data

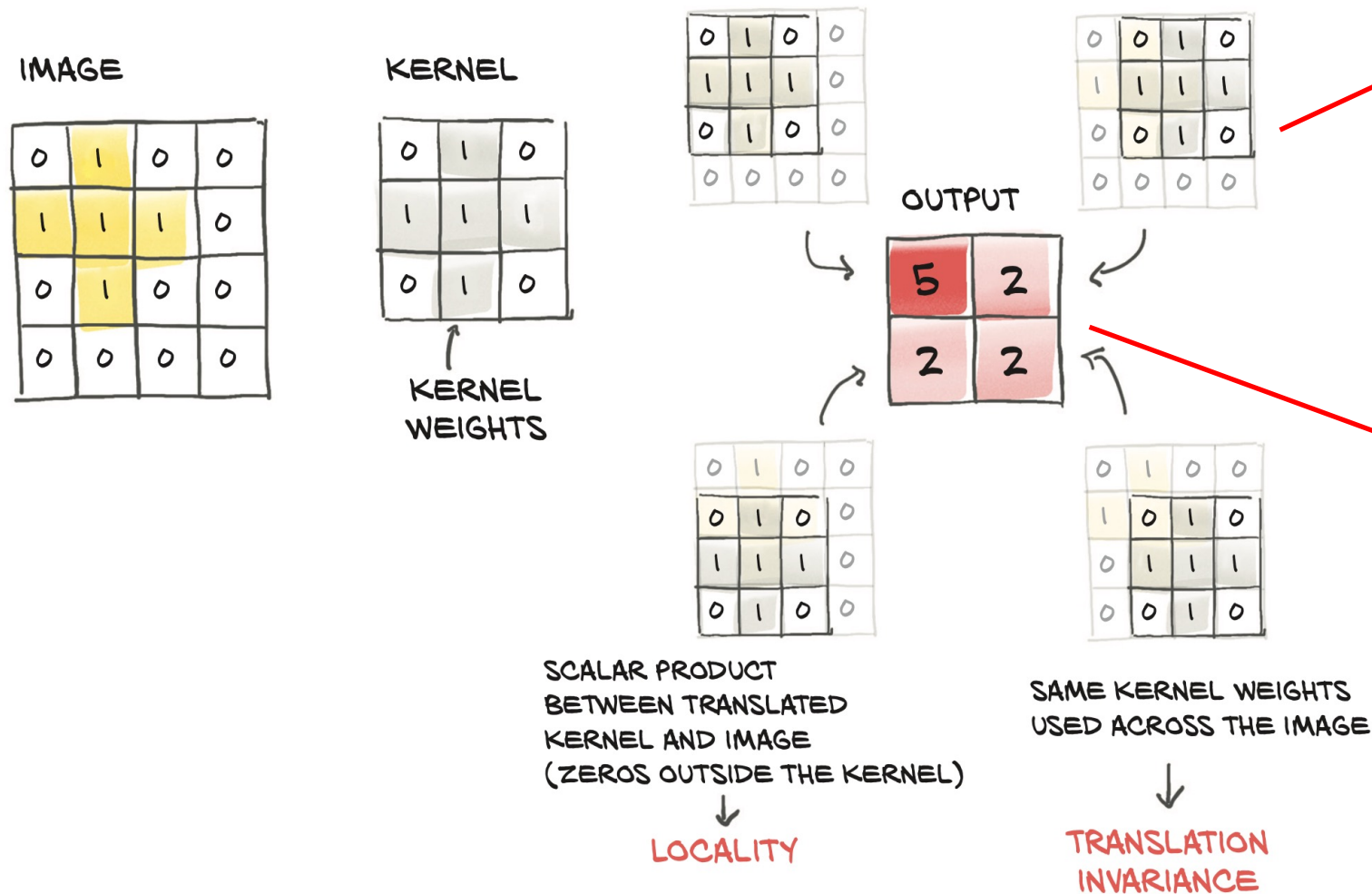
Classification report on train:

precision recall f1-score support

Confusion matrix on train:

Confusion matrix on train:											0	0.99	0.95	0.97	121
[[115 0 1 1 0 1 0 2 1 0]											1	0.85	0.79	0.82	107
[0 84 2 0 0 5 4 9 1 2]											2	0.94	0.80	0.87	128
[0 5 103 1 4 2 0 11 0 2]											3	0.95	0.81	0.87	129
[0 0 0 104 2 0 0 22 0 1]											4	0.94	0.97	0.95	120
[0 0 0 0 116 0 0 4 0 0]											5	0.89	0.98	0.94	120
[1 1 0 0 0 118 0 0 0 0]											6	0.97	0.93	0.95	120
[0 8 0 0 0 0 112 0 0 0]											7	0.68	0.98	0.80	115
[0 0 0 2 0 0 0 113 0 0]											8	0.98	0.91	0.94	107
[0 1 1 0 1 2 0 1 97 4]											9	0.91	0.89	0.90	108
[0 0 3 1 0 4 0 4 0 96]]]															
											ccuracy			0.90	1175
											macro avg			0.91	1175
											weighted avg			0.91	1175

Convolution



- An element-wise multiplication between the image and filter/kernel, and then is summed, resulting in a single number/value.
- The filter is smaller than the input image, which allows the same filter/kernel to be applied to each filter-size patch, left to right, top to bottom.

<https://machinelearningmastery.com/convolutional-layers-for-deep-learning-neural-networks/>

Convolution

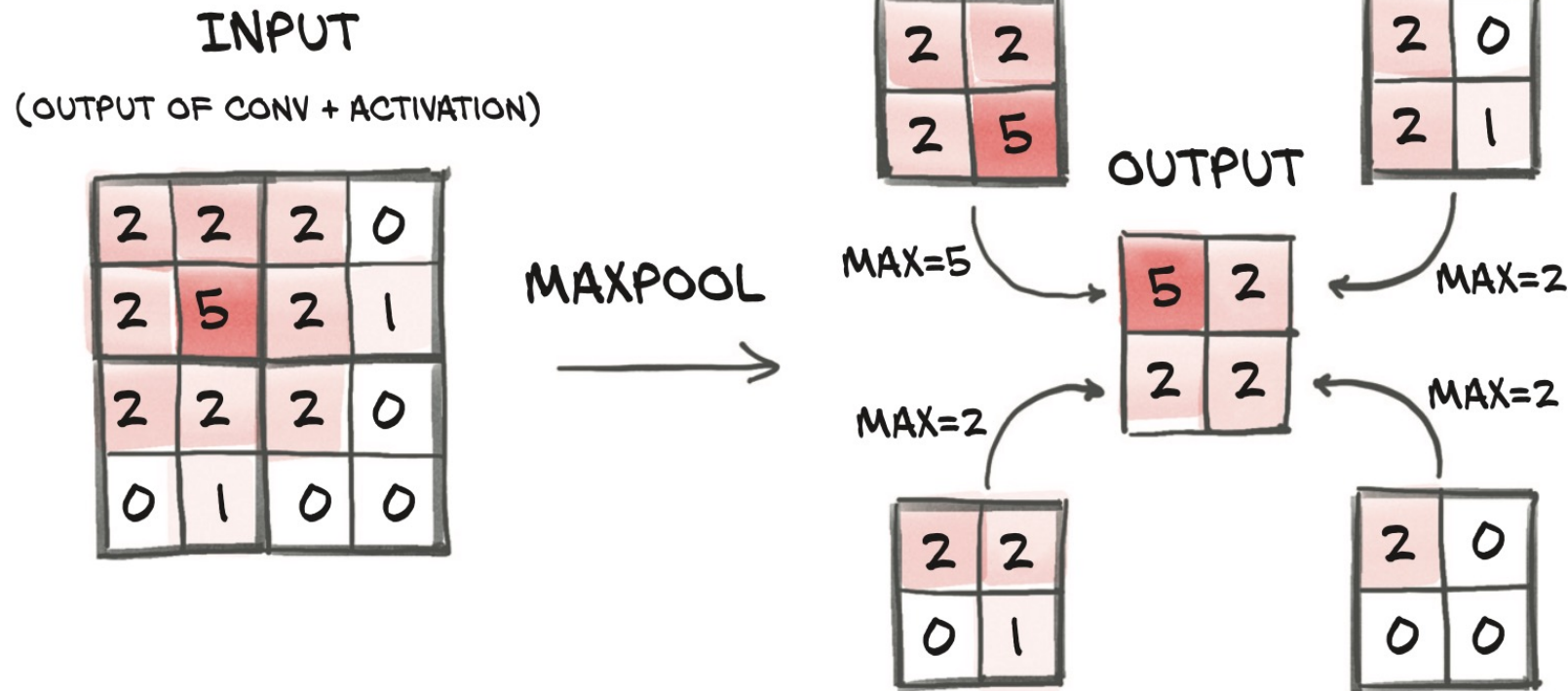
- A convolution is equivalent to having multiple linear operations whose weights are zero almost everywhere except around individual pixels.
 - Models with a lot **fewer parameters**: the number of parameters depends not on the number of pixels in the image, but rather on the size of the convolution kernel and on how many convolution filters.
 - **Local operations** on neighborhoods
 - **Translation invariance**

Benefits of convolutions

Pooling

- The use of small kernels, like 3×3 , or 5×5 : that's peak locality, all right. What about the *big picture*?
- How are our networks going to be equipped to see those patterns with larger scope?
 - Downsampling: scaling an image by half is the equivalent of taking four neighboring pixels as input and producing one pixel as output.
 - *Average pooling: Average the pixels.*
 - *Max pooling: Take the maximum of the pixels*

Pooling



- By keeping the highest value in the 2×2 neighborhood as the downsampled output, we ensure that the features that are found *survive* the downsampling.

Convolutional Neural Network

